# Unity 프로그래밍

## 학습목표

- C# 언어로 프로그램을 작성할 수 있다.

## 개요

## 객체

### 기본 타입

정수 타입은 여기서 볼 수있다. 부동 소수점 타입은 여기서 볼 수 있다. 또, 불리언 타입과 UTF-16 문자 타입을 지원한다. 가장 많이 사용되는 타입은 아래와 같다.

```
// 정수
byte b = 16;            // 10진수
int i = 0x64;           // 16진수
long l = 0b_0010_1010;  // 2진수

// 부동 소수점
float f = 3f;           // float 타입의 소수

// 불리언
bool bo = true;
bo = false;

// UTF-16
char ch = '😁';
```

## 문자열

문자열은 System.String 으로 제공된다. 문자열 활용 방법은 여기를 참고한다. C++과 다르게 C#에서 문자열 타입은 불변(Immutable) 타입이라 항상 새로운 인스턴스를 생성하므로 성능에 주의한다.

```
string s = "Hello ";
s += "World!";
// C++에서는 기존 std::string이 내부적으로 "Hello World!"가 생성되지만
// C#에서는 "Hello World!"라는 새로운 문자열 인스턴스를 만든다.

// 문자열 보간을 이용하면 편하게 형식화된 문자열을 만들 수 있다.
int id = 1234;
s = $"ID는 {id}입니다.";
```

### 산술 연산

C++과 같은 산술 연산자를 제공한다.

```
int a = 10;
int b = 20;

a + b; // 30
a - b; // -10
a * b; // 200
a / b; // 0
a % b; // 10

// 복합 할당 연산
a += b;
a -= b;
a *= b;
a /= b;
a %= b;

// 증감 연산자
++a;
a++;
--a;
a--;
```

### 비트 연산

비트 연산도 지원한다.

```
byte a = 0x20;

~a;      // 0xdf
a << 1; // 0x40
a >> 1; // 0x20

byte b = 0x1f;

a & b;  // 0x10
a | b;  // 0x2f
a ^ b;  // 0xef
```

## 제어문

### 비교 연산

제어문에 들어가는 조건식은 모두 bool 타입이다. 비교 연산은 여기와 여기를 참고한다.

```
int a = 6;
int b = a;

a == b; // true
a != b; // false
a < b;  // false
a > b;  // false
a >= b; // true
a <= b; // true
```

### 분기문

C#도 똑같이 if문과 swtich문을 지원한다. 분기문에 대한 문서는 여기를 참고한다.

#### if문

```
int a = 10;

if (a == 10)
```

```
{
    // 여기가 실행됩니다.
}
else
{
    // 조건식이 거짓일 경우 실행됩니다.
}
```

switch문

```
char ch = 'a';

switch (ch)
{
    case 'A': case 'E': case 'I': case 'O': case 'U':
    case 'a': case 'e': case 'i': case 'o': case 'u':
        Debug.Log("모음임");
        break;

    default:
        Debug.Log("자음임");
        break;
}

// 문자열도 사용 가능
string s = "Hello";
switch (s)
{
    case "Hello":
        Debug.Log("switch 문에서는 문자열 비교 가능!");
        break;
    case "World":
        break;
    default:
        break;
}
```

## 반복문

C++과 마찬가지로 for, do-while, while을 지원하며 break, continue문을 지원한다. 문서는 여기와 여기를 참고한다.

```
for (int i = 0; i < 3; ++i)
{
    // 3번 동안 실행
}

while (true)
{
    // 무한 반복
}

do
{
    // 1번 이상 실행
} while (false);
```

## 배열

C#에서는 C++의 배열보다 훨씬 더 많은 기능을 제공한다. 자세한 것은 여기를 참고한다.

```
// 1차원 배열
int[] arr = new int[5]; // int arr[5]; 와 같다.

// 초기화
arr = new int[5] { 1, 2, 3, 4, 5 };
arr = { 1, 2, 3, 4, 5 };

// 원소 접근
arr[1]; // C++과 같다.

// 다차원 배열
int[,] arr2 = new int[5, 5]; // int arr[5][5];와 같다.

// 초기화
arr2 = new int[5, 5] { { 1, 2, 3 }, { 4, 5, 6 }};
arr2 = { { 1, 2, 3 }, { 4, 5, 6 } };

// 원소 접근
arr[1, 2]; // 문법에 주의한다. 값은 6이다.

// 가변 배열 : 배열의 배열
int[][] arr3 = new int[3][];
```

```
arr3[0] = new int[2] { 1, 2 };
arr3[1] = new int[4] { 1, 2, 3, 4 };
arr3[2] = new int[3] { 1, 2, 3 };

// 원소 접근
arr[1][2]; // 3
```

## 함수

문법은 C++과 같으며 기본 인자와 오버로딩 또한 가능하다.

```
// C++
int Add(int a, int b = 10)
{
    return a + b;
}

int Add(float a, float b = 3.0f)
{
    return a + b;
}

// C#
int Add(int a, int b = 10)
{
    return a + b;
}

int Add(float a, float b = 3f)
{
    return a + b;
}

// 구문이 하나인 경우 아래와 같이 쓸 수 있다.
int Add(int a, int b = 10) => a + b;
```

### 간접 참조

C#의 경우 간접 참조에 좀 더 높은 가독성을 위해 여러 문법을 지원한다. 이를 매개변수 한정자(Parameter Specifier)라고 한다. 여기에는 ref / in / out이 있다.

**ref**

```
// C++
void Swap(int& a, int& b)
{
    int temp = a;
    a = b;
    b = temp;
}

// C#
// ref 한정자를 쓰면 인자가 반드시 초기화 되어 있어야 한다.
// 초기화를 하지 않으면 컴파일 오류가 난다.
void Swap(ref int a, ref int b)
{
    int temp = a;
    a = b;
    b = temp;
}

int a = 10;
int b = 20;
Swap(ref a, ref b); // 호출할 때 꼭 ref 키워드를 적는다.
```

**in**

```
// C++
void Foo(const int& a, const int& b)
{
    // Do Something...
}

// C#
// in 한정자도 인자가 반드시 초기화 되어야 한다.
// 초기화를 하지 않으면 컴파일 오류가 난다.
void Foo(in int a, in int b)
{
    // Do Something...
}
```

**out**

```
// C++
void Foo(int a, int b, int& result)
{
    result = a + b;
}

// C#
// out 한정자는 함수가 끝나기 전 반드시 어떤 값이 할당되어야 한다.
void Foo(int a, int b, out int result)
{
    result = a + b;
}

int r;
Foo(10, 20, out r); // 호출할 땐 꼭 out 키워드를 적는다.
```

## 클래스

이번엔 클래스와 관련된 문법을 살펴본다.

### 기본

C++과 동일하게 필드와 메소드를 작성할 수 있으나, 문법이 다르다. 접근 한정자를 매번 적어야 하며, 접근 한정자의 종류 또한 다르다. 접근 한정자의 종류는 여기를 참고한다.

```
// C++
class Temp
{
public:
    Temp()
    {
        std::cout << "기본 생성자"
    }

    Temp(int a, int b)
        : a(a), b(b)
    {

    }

    Temp(const Temp& other)
```

```
        : a(other.a), b(other.b)
    {

    }

    void Print()
    {
        std::cout << a << ", " << b << "\n";
    }
private:
    int a = 0;
    int b = 0;
};

Temp temp = Temp(1, 2); // C++에서의 객체 생성

// C#
public class Temp
{
    private int a = 0;
    private int b = 0;

    public Temp() => Debug.Log("기본 생성자");

    // 초기자 리스트가 없다.
    public Temp(int a, int b)
    {
        // C#에는 포인터라는 게 없어서 -> 연산자는 없다.
        this.a = a;
        this.b = b;
    }

    // 복사 생성자는 매우 드물게 작성한다.
    public Temp(Temp temp)
    {
        a = temp.a;
        b = temp.b;

        // 얕은 복사인 경우 아래와 같은 메서드를 이용할 수 있다.
        // this = temp.MemberwiseClone();
    }
```

```
    public void Print() => Debug.Log($"{a}, {b}");
}

Temp temp = new Temp(1, 2); // C#에서의 객체 생성
```

## 속성

속성(Property)은 필드의 확장된 버전이라고 생각하면 된다. 속성을 이용하면 좀 더 적은 코드로 필드와 관련된 메서드를 작성할 수 있다. 아래는 속성 사용 예시이다.

```
// C++
class Person
{
public:
    std::string GetFirstName() const { return _firstName;}
    std::string GetSecondName() const { return _secondName; }
    std::string GetFullName() const { return _firstName +
_secondName; }

    void SetFirstName(const std::string& firstName) { _firstName =
firstName; }
    void SetSecondName(const std::string& secondName) { _secondName =
secondName; }
private:
    std::string _firstName;
    std::string _secondName;
}

// C#
class Person
{
    public string FirstName { get; set; }
    public string SecondName { get; set; }
    public string FullName
    {
        get { return $"{FirstName} + {SecondName}"; }
    }
}
```

## 상속과 다형성

상속은 C++과 비슷하나, 접근 한정자를 안 써도 된다.

```
// C++
class Base
{

};

// public을 항상 써줘야 함
class Derived : public Base
{

};

// C#
class Base
{

}

// 안써도 됨
class Derived : Base
{

}
```

또, C#에서는 C++과 다르게 다중 상속을 지원하지 않는다.

```
class Base1 { }
class Base2 { }
class Derived : Base1, Base2 // 컴파일 오류!
{

}
```

## 확장 메서드

확장 메서드(Extension Method)는 기존 타입을 수정하지 않고도 기존 타입에 메서드를 추가할 수 있다. 사용 예제는 여기를 참고한다. 아래는 Unity 엔진에서 문자열을 사용할 때 좀 더 성능이 좋은 커스터마이징 버전의 EndsWith() / StartsWith() 다.

```
public static class Extensions
{
    public static bool CustomEndsWith(this string a, string b)
    {
        int ap = a.Length - 1;
        int bp = b.Length - 1;

        while (ap >= 0 && bp >= 0 && a [ap] == b [bp])
        {
            ap--;
            bp--;
        }

        return (bp < 0);
    }

    public static bool CustomStartsWith(this string a, string b)
    {
        int aLen = a.Length;
        int bLen = b.Length;

        int ap = 0; int bp = 0;

        while (ap < aLen && bp < bLen && a [ap] == b [bp])
        {
            ap++;
            bp++;
        }

        return (bp == bLen);
    }
}
```

## 기호 상수

C#에서 기호상수를 만드는 방법은 아래와 같다.

### const

### readonly

#### 열거형

### const

### #define

#### 열거형

## 제네릭

C++의 템플릿과 같이 C#에서는 제네릭(Generic)이라는 기능이 있다. 간단한 사용 방법은 여기를 참고하자.

#### 제약 조건

타입 파라미터로 전달될 수 있는 타입 인자에 조건을 달 수 있다. 제약 조건을 달면 좀 더 타입에 안전한 코드를 작성할 수 있다. 제약 조건에 대해선 여기를 참고하라. 이와 비슷한 기능으로 C++에서는 concepts라는 것이 있다.

## 컬렉션

C#에서도 다양한 자료구조를 제공한다. C++에서 사용하던 STL 컨테이너와 매칭되는 클래스는 아래와 같다.

- unordered_map : Dictionary
- unordered_set : HashSet
- list : LinkedList
- vector : List
- priority_queue : PriorityQueue
- queue : Queue
- map : SortedDictionary
- set : SortedSet
- stack : Stack

## 수학 라이브러리

수학과 관련하여 Unity에서는 Mathf라는 클래스를 제공하고 있다.

## 시간 라이브러리

시간과 관련하여 Unity에서는 Time 클래스를 제공하고 있다.

# 부록

Unity와 관련하여 읽어볼만한 여러 가지 자료를 제공한다.

- Awake / OnEnable / Start의 차이
- Unity에 싱글톤 패턴 적용하기

## 참고자료

- C# 스터디
- C# 문서 - 시작, 자습서, 참조.